

1. Introduction

The Medical Inventory Management System, branded as MediCore Hospital Admin, is a standalone desktop application developed in Java using the JavaFX framework. It is designed to assist hospital administrators and pharmacy managers in efficiently managing their medicine inventory, tracking stock movements, monitoring expiry dates, and generating comprehensive reports.

In modern hospitals and medical facilities, the management of pharmaceutical inventory is a critical task that demands accuracy, timeliness, and accountability. Manual tracking systems or spreadsheet-based approaches often lead to errors, stock shortages, expired medicines being dispensed, and loss of financial control. MediCore addresses all these challenges by providing a centralized, user-friendly digital platform. The application provides a comprehensive suite of features including:

- Secure administrator authentication with registration and login capabilities.
- Full CRUD (Create, Read, Update, Delete) operations on medicine records.
- Real-time stock movement tracking for stock IN and stock OUT transactions.
- Automatic alerts for low-stock medicines and near-expiry products.
- Dashboard with statistics, bar charts, and demand analysis.
- Export capabilities to generate CSV and PDF reports for auditing and compliance.
- Dark mode support and configurable low-stock threshold settings.

The system stores all data locally using SQLite, making it suitable for small to medium-sized medical facilities that may not have access to a centralized server infrastructure. The application follows the MVC (Model-View-Controller) architectural pattern and employs a repository pattern for data access, ensuring clean separation of concerns and maintainability.

2. Problem in Existing System

The existing medicine inventory management in most small hospitals and clinics relies on manual processes, paper-based records, or basic spreadsheet software such as Microsoft Excel. These traditional systems suffer from numerous critical limitations:

2.1 Manual Record Keeping Errors

Manual data entry is highly prone to human error. Pharmacists may record incorrect quantities, misplace entries, or fail to update records promptly after dispensing medicines. Such errors can lead to incorrect stock counts, leading to either over-ordering or critical shortages.

2.2 Difficulty in Tracking Expiry Dates

In paper-based systems, tracking expiry dates of hundreds of medicines simultaneously is virtually impossible. Expired medicines may go unnoticed and be dispensed to patients, posing serious health risks and legal liabilities to the institution.

2.3 Inefficient Reporting

Generating reports from manual systems requires significant effort and time. Compiling data for monthly audits, supply chain meetings, or regulatory compliance is a tedious process that diverts staff from core medical duties.

2.4 Data Security and Access Control

Manual systems offer no access control mechanisms. Any staff member with physical access to records can view, modify, or even destroy critical inventory data, posing significant security and compliance risks.

3. Need for System

The development of the Medical Inventory Management System (MediCore) was motivated by the pressing need to modernize and digitize the medicine inventory management process in healthcare facilities. The following key factors underscore the necessity of such a system:

3.1 Patient Safety

Accurate medicine inventory management is directly linked to patient safety. Ensuring that medicines are available when needed, are not expired, and are dispensed in correct quantities is critical. An automated system with expiry alerts and stock monitoring directly contributes to safer patient care.

3.2 Operational Efficiency

Hospital staff spends a significant amount of time on inventory-related tasks. A computerized system automates repetitive tasks such as stock calculation, report generation, and alert generation, freeing staff to focus on direct patient care activities.

3.3 Financial Control

Medicines represent a major portion of a hospital's operating budget. Effective inventory management reduces wastage from expired medicines, prevents over-purchasing, and enables data-driven procurement decisions. The dashboard's inventory value calculation helps administrators maintain financial oversight.

3.4 Regulatory Compliance

Healthcare facilities are required to maintain proper records of pharmaceutical inventory for regulatory audits and inspections. The system's ability to generate PDF and CSV reports ensures that compliance documentation is always readily available.

3.5 Scalability and Reliability

Unlike spreadsheet-based solutions that become unwieldy as data grows, a database-backed application scales effectively. SQLite ensures reliable data persistence while remaining lightweight and suitable for standalone deployment.

4. Scope of Proposed System

- **Administrator Registration and Authentication:** Single-admin system with secure password hashing and session management.
- **Medicine Inventory Management:** Complete CRUD operations for medicine records including name, quantity, price, and expiry date.
- **Stock Movement Tracking:** Recording of all stock IN (restock) and stock OUT (consumption) transactions with reference numbers and notes.
- **Automated Alerts and Notifications:** System-generated alerts for medicines with stock below the configured threshold and medicines expiring within 30 days.
- **Dashboard and Analytics:** Real-time statistics showing total medicines, low-stock count, near-expiry count, expired medicines, and total inventory value with graphical demand analysis.
- **Report Generation:** Export of current stock, near-expiry medicines, and stock movement reports in both CSV and PDF formats.
- **System Settings:** Configurable low-stock threshold, dark mode toggle, and notification preferences.

5. Feasibility Study & Fact-Finding Technique Applied

5.1 Feasibility Study

Technical Feasibility: The system is built using Java 24 with JavaFX for the graphical user interface. Both technologies are mature, well-documented, and freely available. SQLite is used as the embedded database, eliminating the need for a separate database server. The open-source Apache PDFBox library is used for PDF generation. All dependencies are managed via Gradle, ensuring reproducible builds. The technical stack is entirely feasible for a single-developer project.

Operational Feasibility: The system has been designed with a simple and intuitive graphical interface to minimize the learning curve for hospital administrative staff. The application requires no network connectivity or external server, making it operationally straightforward to deploy and maintain.

Economic Feasibility: All technologies used in the system are open-source and free of cost. The application runs on any standard desktop or laptop computer. The economic cost of development is primarily the time investment, making it highly economically feasible for educational and institutional deployment.

5.2 Fact-Finding Techniques Applied

The following fact-finding methods were employed during the requirements analysis phase:

- **Interview:** Discussions were held with pharmacy staff and hospital administrators to understand their daily workflow, pain points, and requirements for inventory management.
- **Observation:** The existing manual stock management process was observed to identify specific inefficiencies and data flow patterns.
- **Document Analysis:** Existing stock registers, medicine purchase orders, and consumption records were analyzed to understand the data elements required.
- **Questionnaire:** A structured questionnaire was administered to gather quantitative data about the volume of medicine transactions and reporting frequency.

6. Hardware and Software Requirement

6.1 Hardware Requirements

The following minimum hardware configuration is required to run the MediCore application:

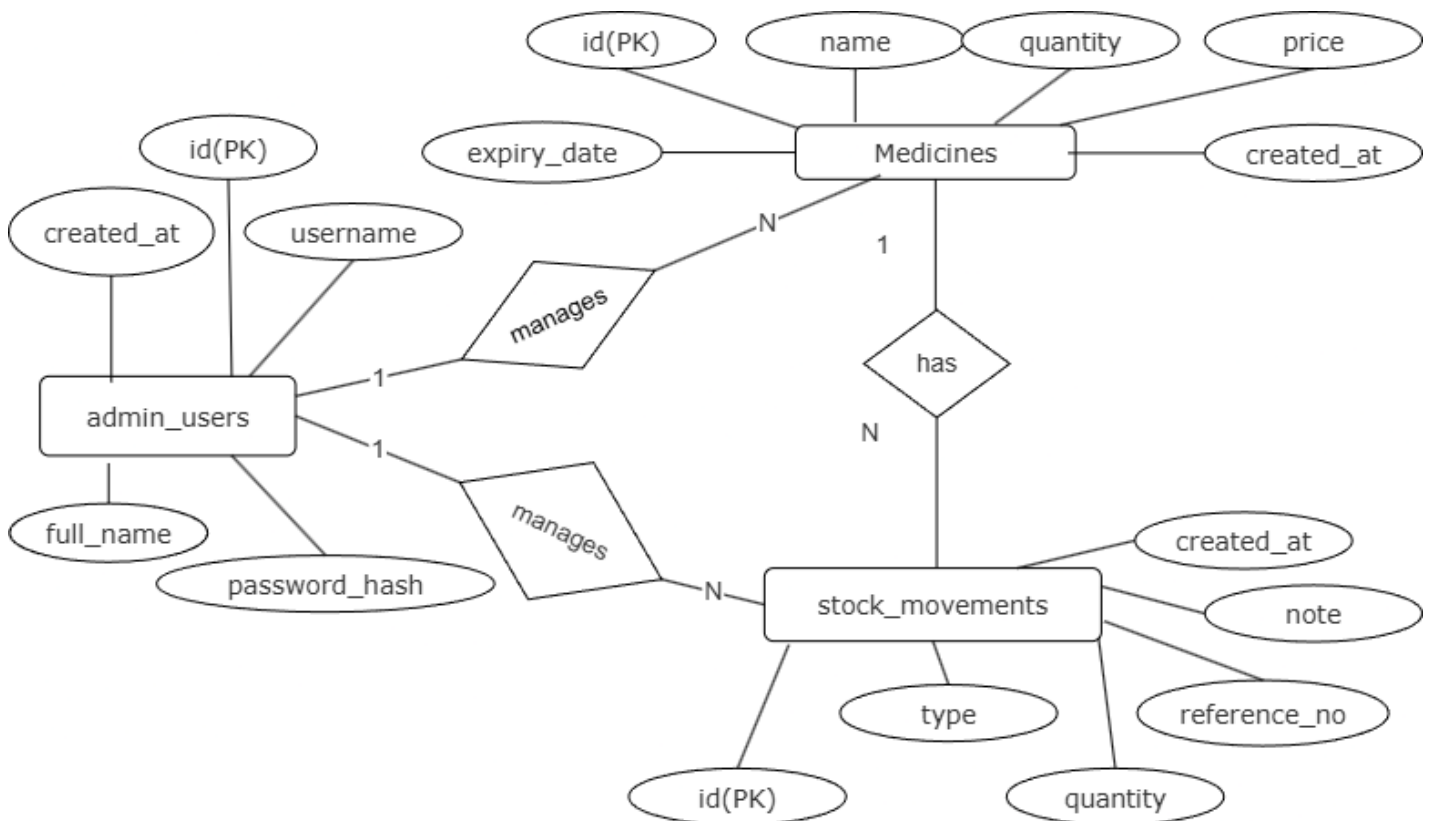
Component	Specification
Processor	Intel Core i3 or AMD equivalent (1.6 GHz or faster)
RAM	Minimum 4 GB (8 GB recommended)
Storage	Minimum 500 MB free disk space
Display	1280 x 720 resolution or higher
Operating System	Windows 10/11, Linux (Ubuntu 20.04+), or macOS 11+

6.2 Software Requirements

Software	Details
Java Development Kit	JDK 24 (OpenJDK or Oracle JDK)
JavaFX SDK	JavaFX 21.0.5
SQLite JDBC Driver	sqlite-jdbc 3.46.1.3 (Xerial)
PDF Library	Apache PDFBox 2.0.31
Build Tool	Gradle 8.x with JavaFX Plugin 0.1.0
IDE (Development)	IntelliJ IDEA / Eclipse / VS Code with Java Extension
Database	SQLite (embedded, no separate installation required)

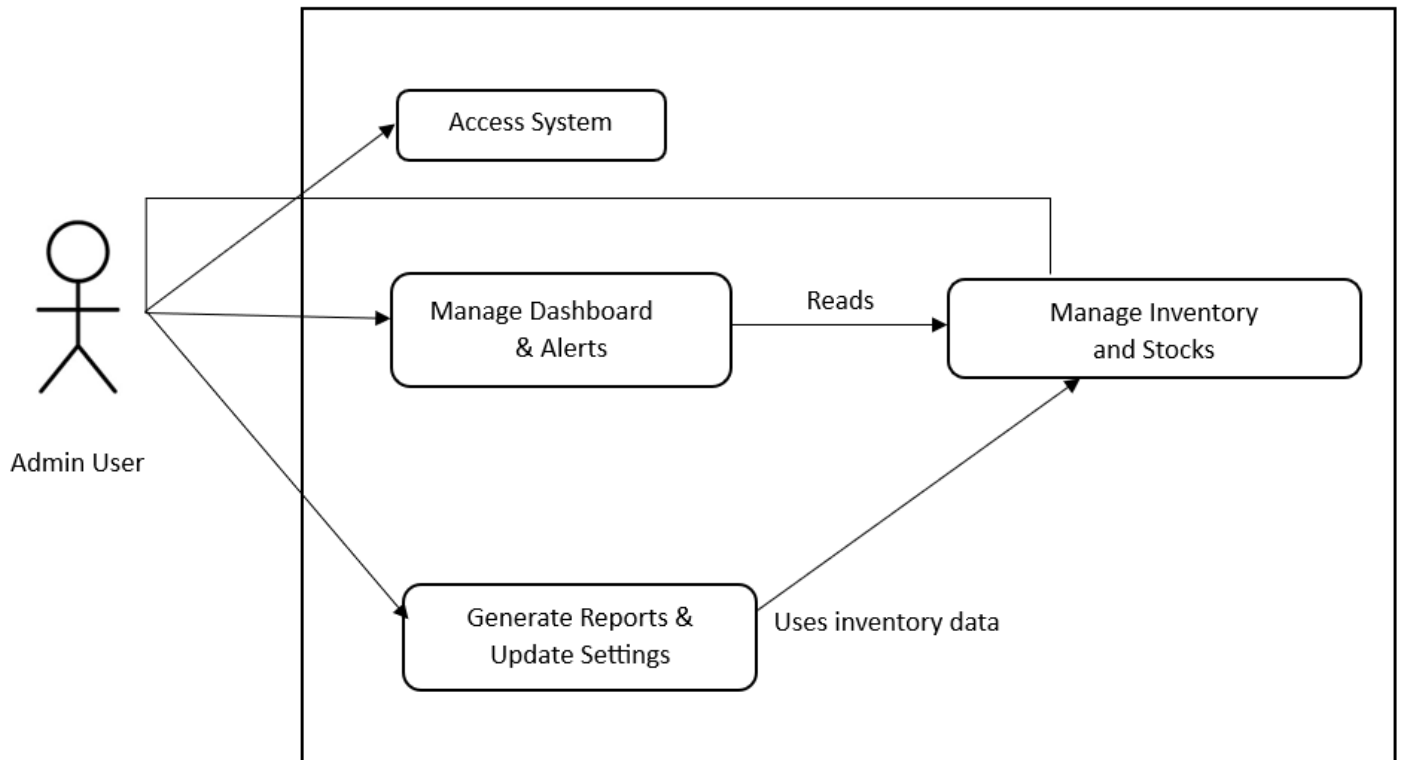
7. Design Specification

7.1 ER Diagram

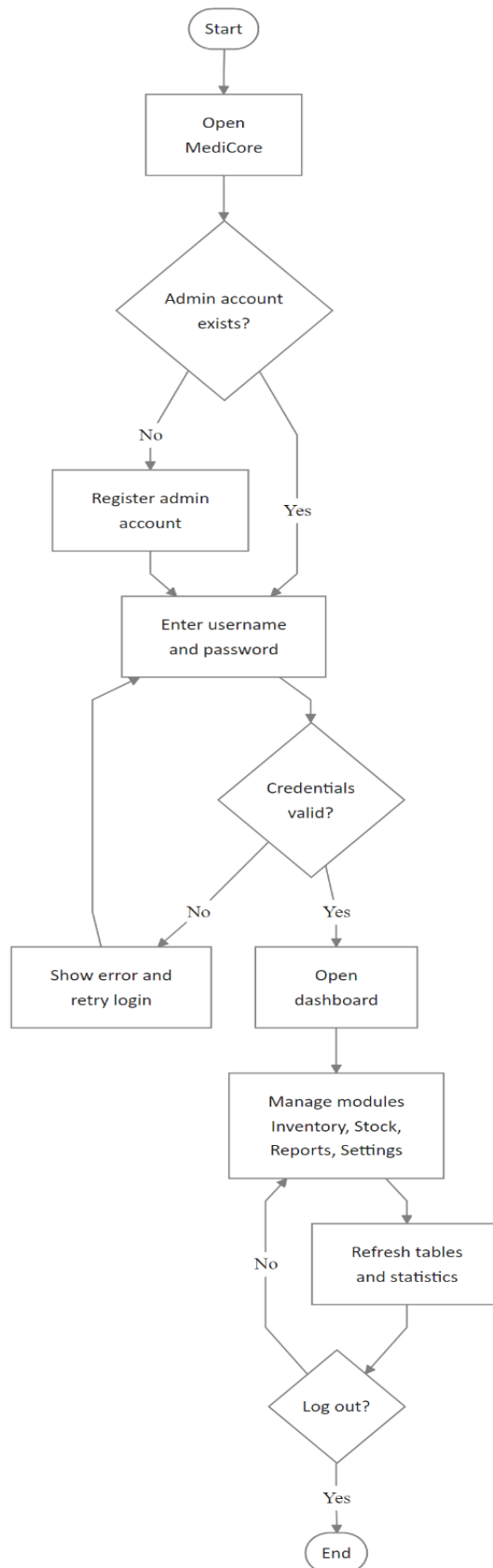


7.2 UML Diagrams

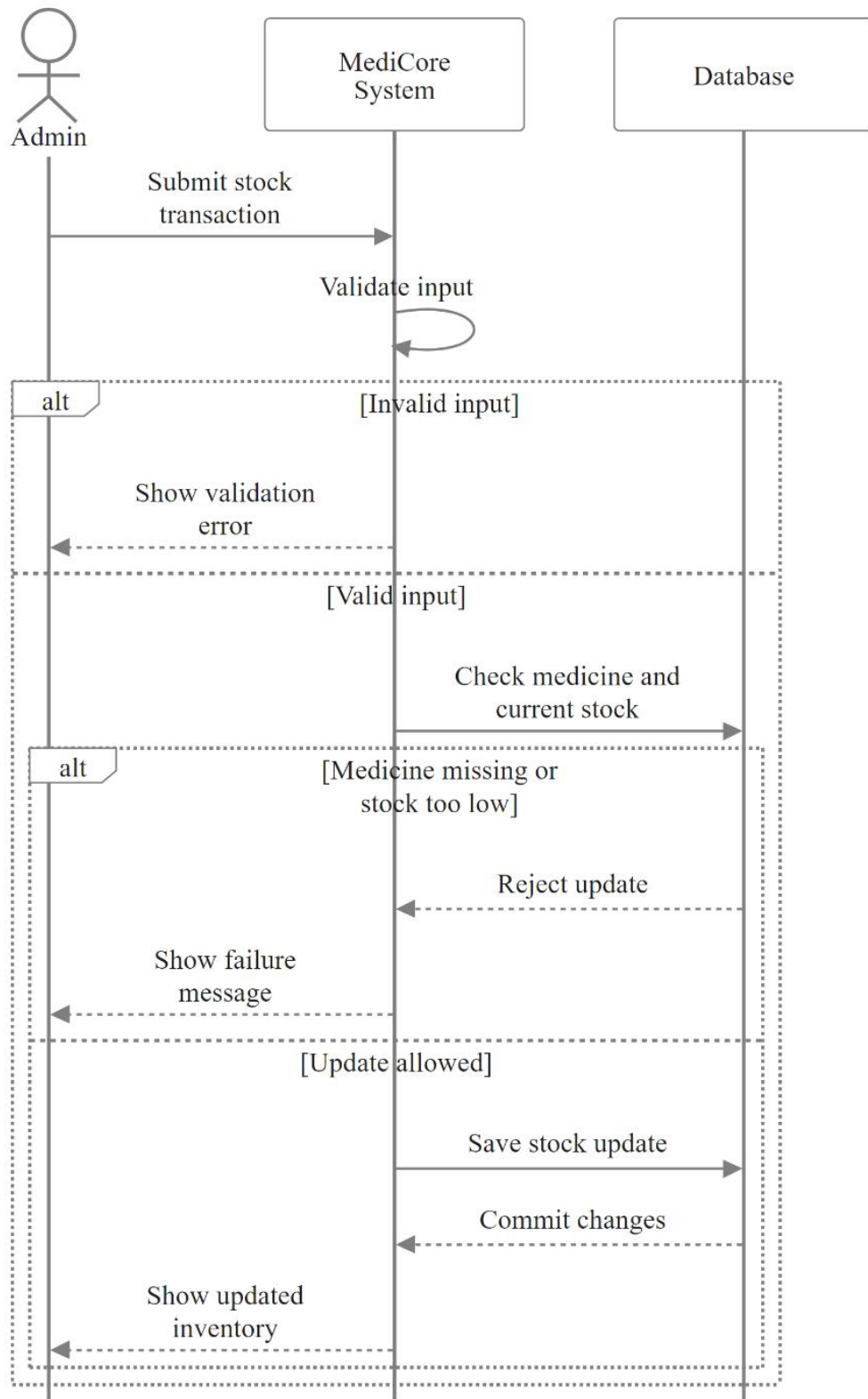
7.2.1 Use Case Diagram



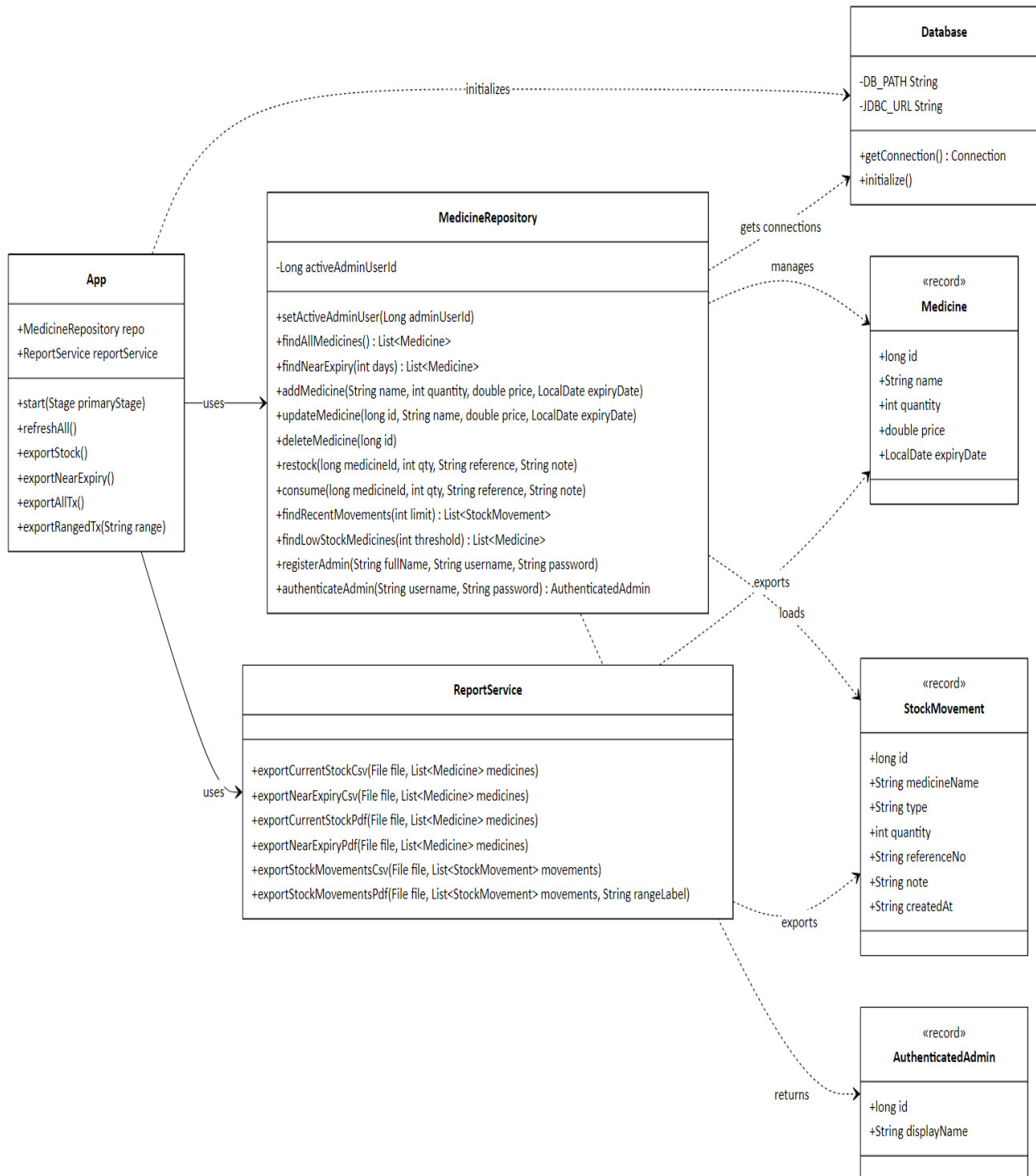
7.2.2 Activity Diagram



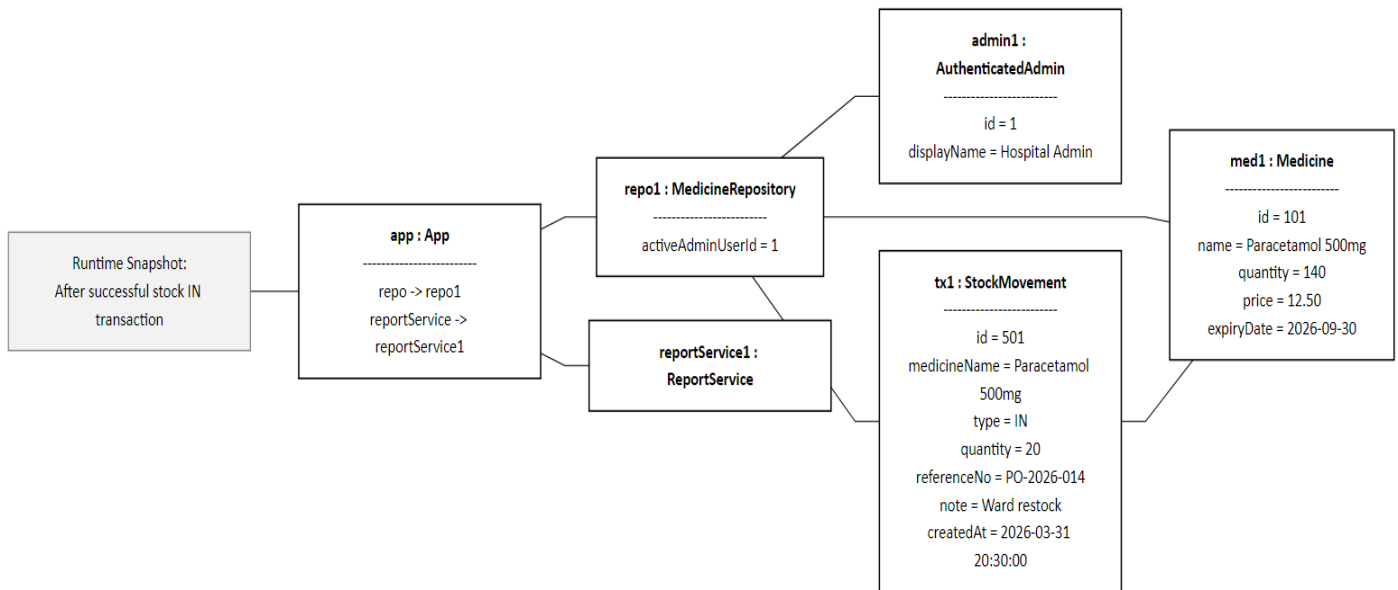
7.2.3 Sequence Diagram



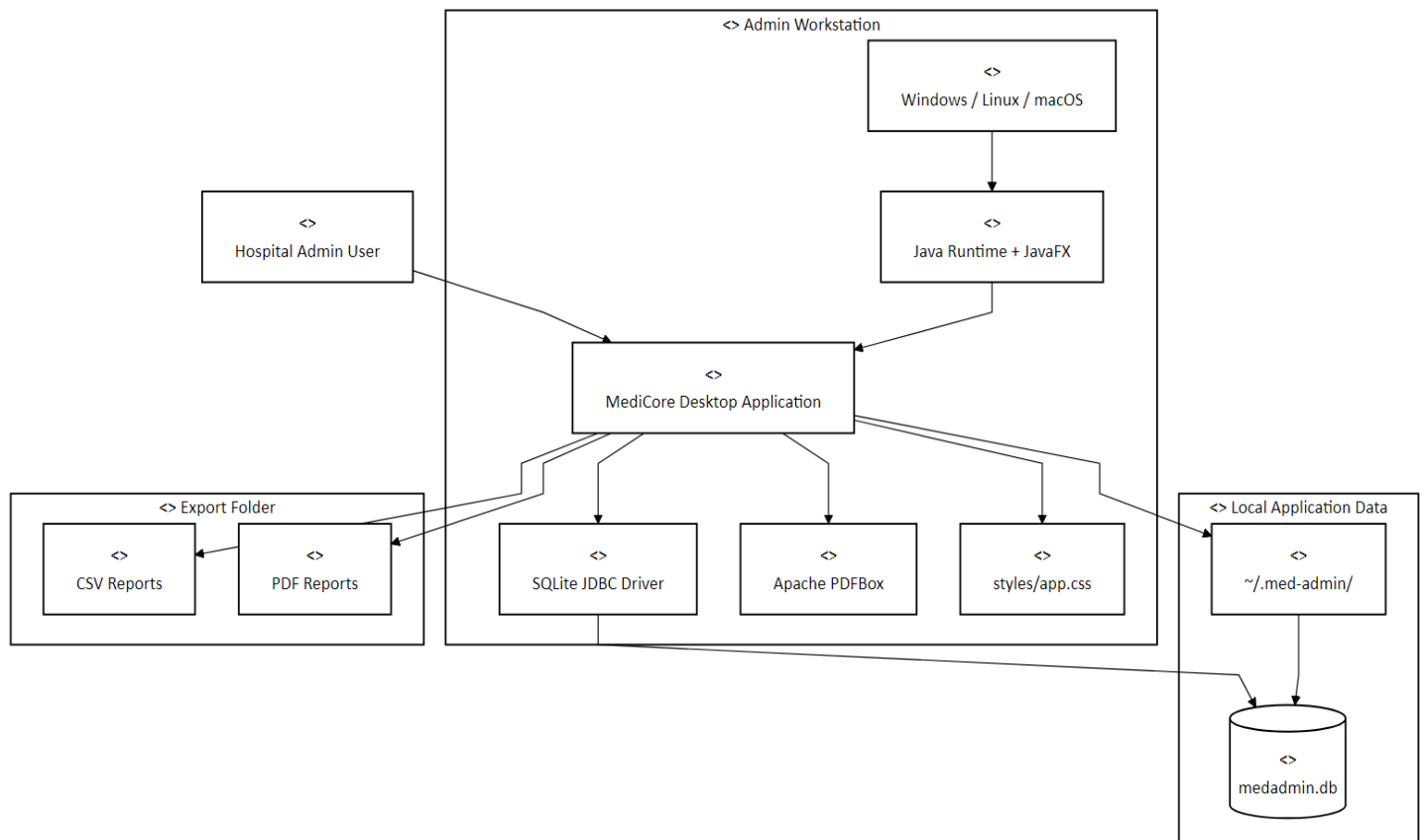
7.2.4 Class Diagram



7.2.5 Object Diagram



7.2.6 Deployment Diagram



8. Data Dictionary

Table 8.1: admin_users

Sr. no	Field Name	Data Type	Size	Description	Table Name
1.	id	INTEGER	4 bytes	Unique identifier for each administrator	admin_users
2.	username	TEXT	Variable	Login username for the administrator	admin_users
3.	password_hash	TEXT	Variable	SHA-256 hashed password	admin_users
4.	full_name	TEXT	Variable	Full name of the administrator	admin_users
5.	created_at	TEXT	Variable	Timestamp of account creation	admin_users

Table 8.2: medicines

Sr. no.	Field Name	Data Type	Size	Description	Table Name
1.	id	INTEGER	4 bytes	Unique identifier for each medicine	medicines
2.	name	TEXT	Variable	Name of the medicine	medicines
3.	quantity	INTEGER	4 bytes	Current stock quantity in units	medicines
4.	price	REAL	8 bytes	Unit price of the medicine	medicines
5.	expiry_date	TEXT	10 chars	Expiry date in YYYY-MM-DD format	medicines
6.	created_at	TEXT	Variable	Record creation timestamp	medicines
7.	admin_user_id	INTEGER	4 bytes	FK reference to admin_users	medicines

Table 8.3: stock movements

Sr.no.	Field Name	Data Type	Size	Description	Table Name
1.	id	INTEGER	4 bytes	Unique identifier for each movement	stock_movements
2.	medicine_id	INTEGER	4 bytes	FK reference to medicines table	stock_movements
3.	type	TEXT	Variable	Transaction type: IN or OUT	stock_movements
4.	quantity	INTEGER	4 bytes	Number of units moved	stock_movements
5.	reference_no	TEXT	Variable	Reference/invoice number	stock_movements
6.	note	TEXT	Variable	Additional notes for the transaction	stock_movements
7.	created_at	TEXT	Variable	Transaction timestamp	stock_movements
8.	admin_user_id	INTEGER	4 bytes	FK reference to admin_users	stock_movements

Table 8.4: app_settings

Sr.no.	Field Name	Data Type	Size	Description	Table Name
1.	id	INTEGER	4 bytes	Singleton record identifier (always = 1)	app_settings
2.	notify_low_stock	INTEGER	1 byte	Flag to enable/disable low stock alerts	app_settings
3.	low_stock_threshold	INTEGER	4 bytes	Quantity below which stock is considered low	app_settings
4.	dark_mode	INTEGER	1 byte	Flag for dark mode (1=enabled)	app_settings
5.	updated_at	TEXT	Variable	Timestamp of last settings update	app_settings

9. Table Design

Table 8.1: admin_users

Sr.no.	Field Name	Data Type	Size	Constraints
1.	id	INTEGER	4 bytes	PRIMARY KEY, AUTOINCREMENT
2.	username	TEXT	Variable	UNIQUE, NOT NULL
3.	password_hash	TEXT	Variable	NOT NULL
4.	full_name	TEXT	Variable	NULLABLE
5.	created_at	TEXT	Variable	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Table 8.2: medicines

Sr.no.	Field Name	Data Type	Size	Constraints
1.	id	INTEGER	4 bytes	PRIMARY KEY, AUTOINCREMENT
2.	name	TEXT	Variable	NOT NULL
3.	quantity	INTEGER	4 bytes	NOT NULL, DEFAULT 0
4.	price	REAL	8 bytes	NOT NULL
5.	expiry_date	TEXT	10 chars	NOT NULL
6.	created_at	TEXT	Variable	NOT NULL
7.	admin_user_id	INTEGER	4 bytes	FOREIGN KEY

Table 8.3: stock movements

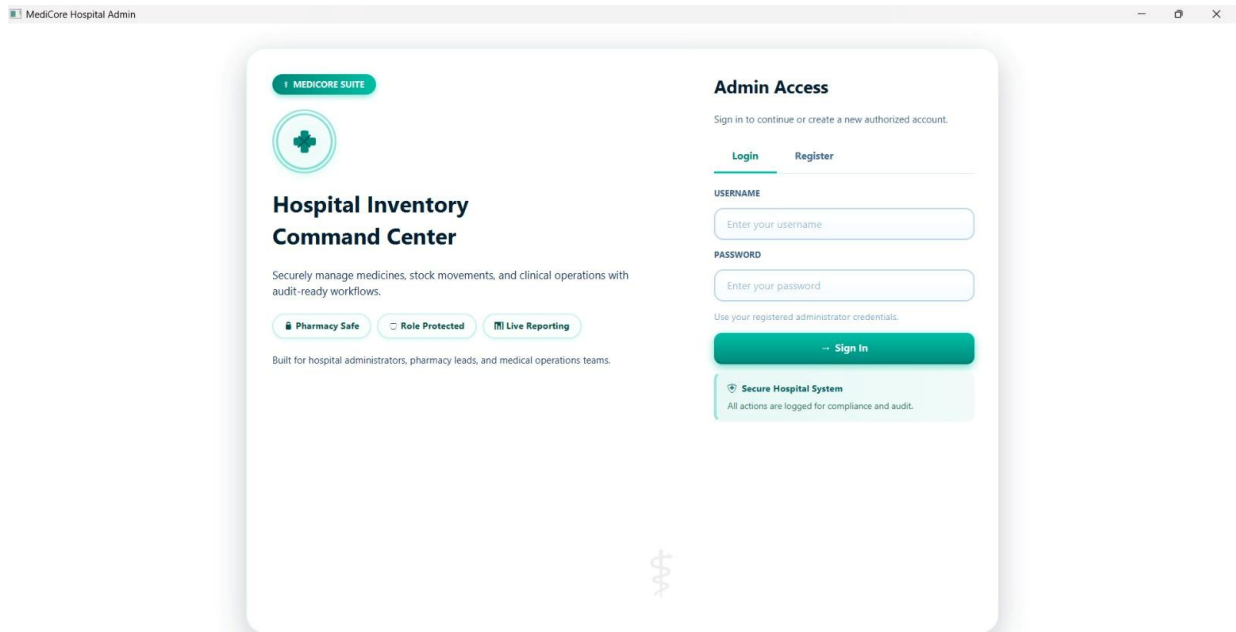
Sr no.	Field Name	Data Type	Size	Constraints
1.	id	INTEGER	4 bytes	PRIMARY KEY, AUTOINCREMENT
2.	medicine_id	INTEGER	4 bytes	NOT NULL, FOREIGN KEY
3.	type	TEXT	Variable	NOT NULL
4.	quantity	INTEGER	4 bytes	NOT NULL
5.	reference_no	TEXT	Variable	NULLABLE
6.	note	TEXT	Variable	NULLABLE
7.	created_at	TEXT	Variable	NOT NULL
8.	admin_user_id	INTEGER	4 bytes	FOREIGN KEY

Table 8.4: app_settings

Sr.no.	Field Name	Data Type	Size	Constraints
1.	id	INTEGER	4 bytes	PRIMARY KEY, CHECK (id=1)
2.	notify_low_stock	INTEGER	1 byte	NOT NULL, DEFAULT 1
3.	low_stock_threshold	INTEGER	4 bytes	NOT NULL, DEFAULT 10
4.	dark_mode	INTEGER	1 byte	NOT NULL, DEFAULT 0
5.	updated_at	TEXT	Variable	NOT NULL

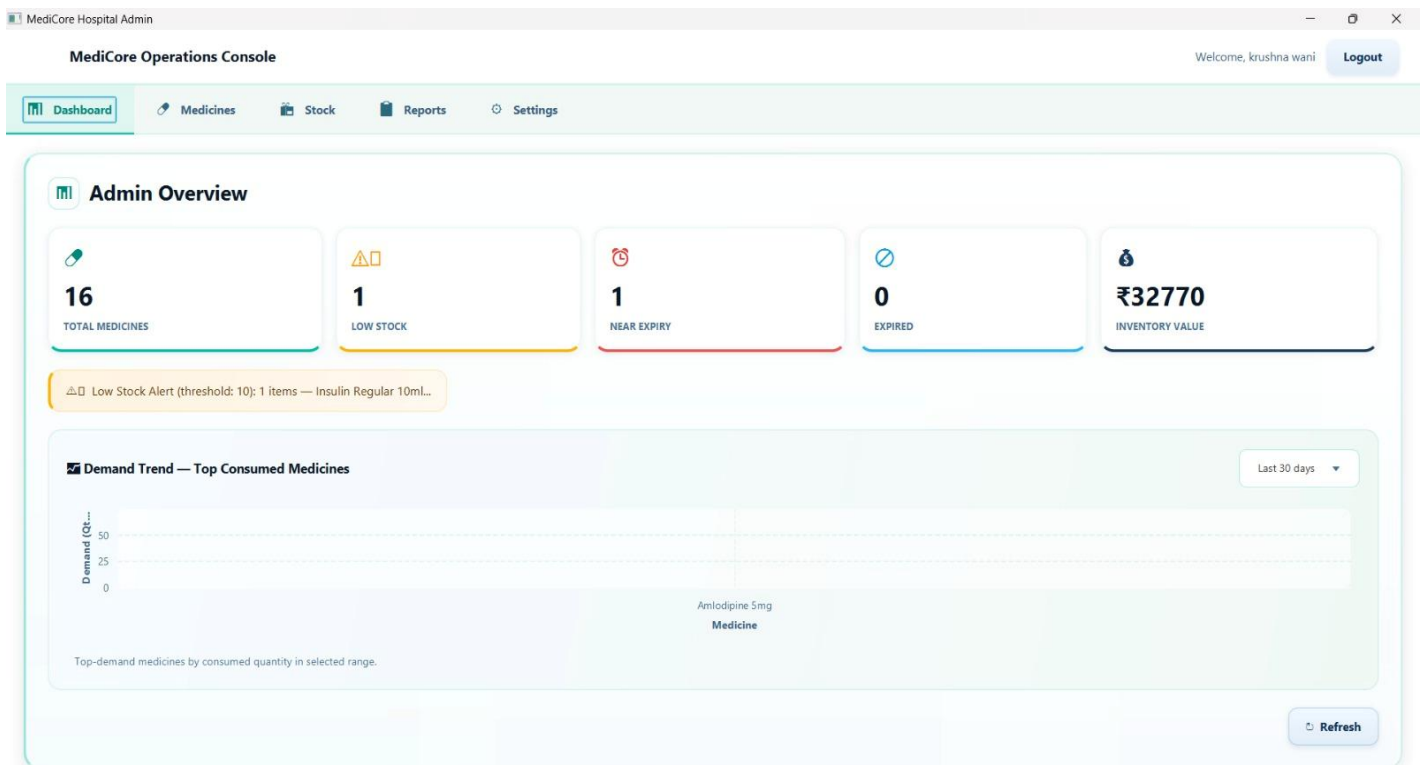
10. Sample Input & Output Screen

10.1 Authentication Screen



The authentication screen is titled "MediCore Hospital Admin" in the browser window. It features a central card with the "MEDICORE SUITE" logo and a green cross icon. The main heading is "Hospital Inventory Command Center". Below this, a description states: "Securely manage medicines, stock movements, and clinical operations with audit-ready workflows." There are three status indicators: "Pharmacy Safe" (green), "Role Protected" (yellow), and "Live Reporting" (blue). A note at the bottom says "Built for hospital administrators, pharmacy leads, and medical operations teams." On the right, the "Admin Access" section includes a sign-in prompt, "Login" and "Register" tabs, and input fields for "USERNAME" and "PASSWORD". A "Sign In" button is at the bottom of the form. A "Secure Hospital System" badge is also present.

10.2 Main Dashboard



10.3 Inventory Management Screen

MediCore Hospital Admin

MediCore Operations Console

Welcome, krushna wani

Logout

Dashboard

Medicines

Stock

Reports

Settings

Medicine Inventory

+ ADD NEW MEDICINE

MEDICINE NAME

e.g. Paracetamol 500mg

PRICE (₹)

0.00

INITIAL QUANTITY

0

EXPIRY DATE

30/09/2026

+ Add Medicine

Edit Selected

Delete Selected

Find

Search medicine by name...

Search

Clear

ID	NAME	QUANTITY	PRICE (₹)	EXPIRY	STATUS
9	Amlodipine 5mg	45	2.9	2027-09-18	✓ Available
4	Amoxicillin 250mg	90	6.75	2026-05-01	✓ Available
10	Atorvastatin 10mg	115	5.2	2027-07-07	✓ Available
5	Azithromycin 500mg	60	14.2	2027-03-20	✓ Available

10.4 Stock Movement Screen

MediCore Hospital Admin

MediCore Operations Console

Welcome, krushna wani

Logout

Dashboard

Medicines

Stock

Reports

Settings

Stock Movements

NEW TRANSACTION

SELECT MEDICINE

Amlodipine 5mg (Qty: 45)

TRANSACTION TYPE

IN OUT

QUANTITY

0

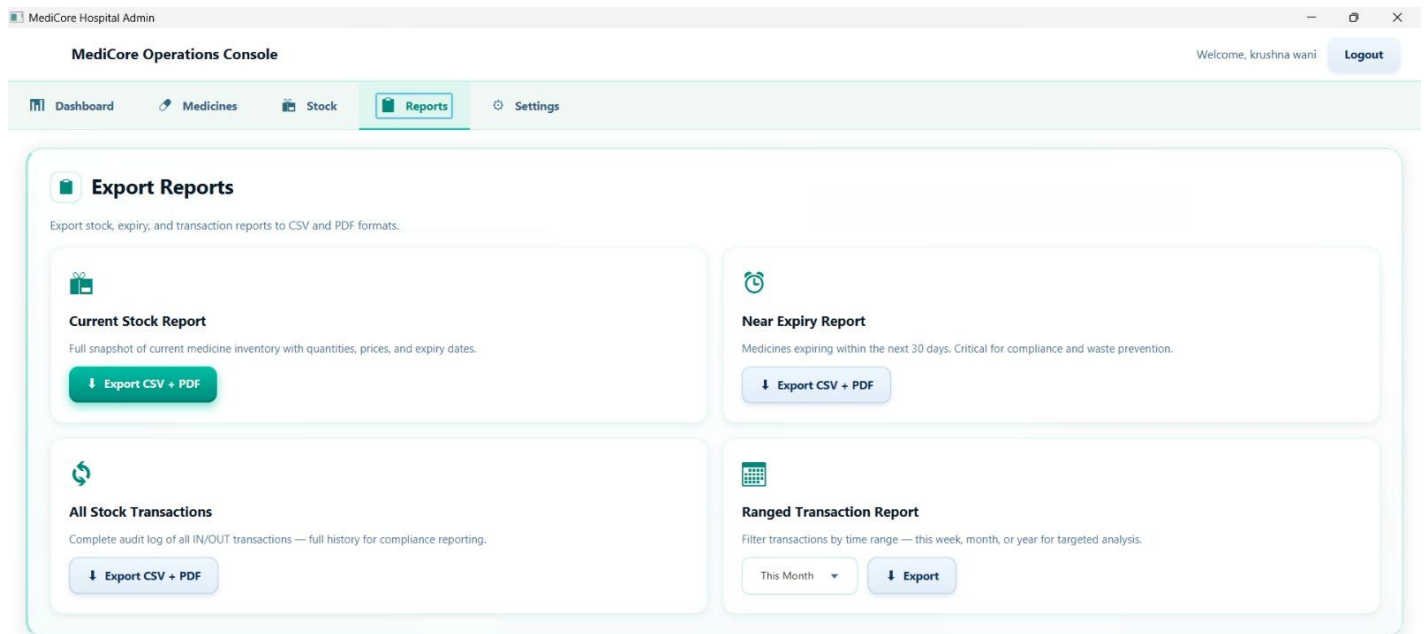
NOTE

Optional note

Save Transaction

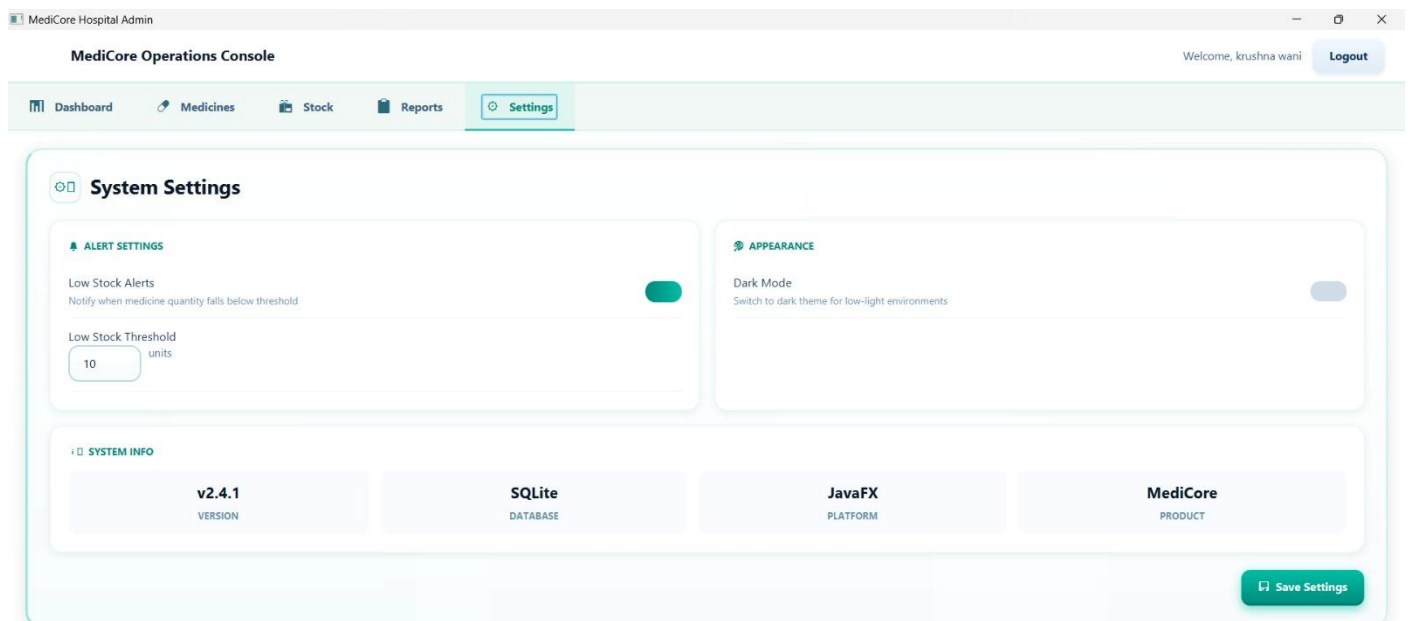
TX ID	MEDICINE	TYPE	QTY	REFERENCE	NOTE	CREATED AT
9	Atorvastatin 10mg	IN	10	—	—	2026-03-27 17:19:30
8	Amlodipine 5mg	IN	20	—	received from another med	2026-03-27 17:18:42
7	Amlodipine 5mg	OUT	20	124	sold	2026-03-26 07:12:47
6	Amlodipine 5mg	OUT	50	123	sold	2026-03-03 09:10:38
5	Insulin Regular 10ml	OUT	40	122	sold	2026-02-28 08:42:13

10.5 Reports Screen



The screenshot shows the 'MediCore Hospital Admin' window with the 'MediCore Operations Console' header. The 'Reports' tab is selected in the navigation bar. The main content area is titled 'Export Reports' and includes a sub-header 'Export stock, expiry, and transaction reports to CSV and PDF formats.' Below this, there are four report cards: 'Current Stock Report' (with an 'Export CSV + PDF' button), 'Near Expiry Report' (with an 'Export CSV + PDF' button), 'All Stock Transactions' (with an 'Export CSV + PDF' button), and 'Ranged Transaction Report' (with a 'This Month' dropdown and an 'Export' button).

10.6 Settings Screen



The screenshot shows the 'MediCore Hospital Admin' window with the 'MediCore Operations Console' header. The 'Settings' tab is selected in the navigation bar. The main content area is titled 'System Settings' and includes two sections: 'ALERT SETTINGS' and 'APPEARANCE'. The 'ALERT SETTINGS' section has a 'Low Stock Alerts' toggle (turned on) and a 'Low Stock Threshold' input field set to '10 units'. The 'APPEARANCE' section has a 'Dark Mode' toggle (turned off). Below these sections is a 'SYSTEM INFO' section with four cards: 'v2.4.1 VERSION', 'SQLite DATABASE', 'JavaFX PLATFORM', and 'MediCore PRODUCT'. A 'Save Settings' button is located at the bottom right.

11. Testing Strategy

A comprehensive testing strategy was applied to ensure the correctness, reliability, and robustness of the MediCore application. The following testing approaches were employed:

11.1 Unit Testing

Individual methods in the MedicineRepository and ReportService classes were tested in isolation to verify that each function produces correct outputs for given inputs. Key unit tests included:

- **Authentication:** Testing with valid credentials (expected: admin ID returned), invalid password (expected: empty Optional), and non-existent username (expected: empty Optional).
- **Medicine CRUD:** Verifying that addMedicine() correctly inserts a record and that the new ID is returned. Testing updateMedicine() for correct field modification. Confirming deleteMedicine() removes the record.
- **Stock Operations:** Testing restock() to verify quantity increases correctly. Testing consume() to confirm quantity decreases and that an exception is thrown when attempting to consume more than available stock.
- **Report Generation:** Validating that CSV files are generated with correct headers and data rows, and that PDF files are created with properly formatted content.

11.2 Integration Testing

Integration tests verified the interaction between the application layers. The JavaFX UI was tested in conjunction with the MedicineRepository to ensure that user actions correctly trigger database operations and that UI components refresh properly after data modifications. The database initialization sequence was tested to confirm that all tables are created correctly on first run.

11.3 System Testing

Complete end-to-end system testing was performed by simulating real-world usage scenarios:

- **Scenario 1 - New Installation:** Fresh application launch, admin registration, and initial medicine addition.
- **Scenario 2 - Stock Management:** Adding multiple medicines, performing restock and consumption transactions, and verifying dashboard statistics update correctly.
- **Scenario 3 - Alert Generation:** Setting low-stock threshold, reducing medicine stock below threshold, and confirming alert banner displays the correct medicine names.
- **Scenario 4 - Report Export:** Generating all three report types in both CSV and PDF formats and verifying file contents match system data.
- **Scenario 5 - Dark Mode:** Toggling dark mode and confirming all UI elements render correctly in both themes.

11.4 Boundary Testing

Boundary conditions were tested to ensure system stability at edge cases:

- **Zero Quantity:** Attempting to consume stock when quantity is zero (expected: "Insufficient stock" error).
- **Expiry Date Boundary:** Testing near-expiry alerts exactly at 30 days, 29 days, and 31 days from current date.
- **Empty Medicine List:** Testing dashboard behavior when no medicines are registered.
- **Duplicate Username:** Attempting to register with an existing username (expected: error dialog).

13. Limitations

While MediCore provides a comprehensive solution for medical inventory management, the current version has several known limitations that should be considered when deploying the system:

- **Single-User Architecture:** The system is designed for a single administrator per installation. There is no support for concurrent multi-user access, role-based access control (RBAC), or multi-level permissions (e.g., pharmacist vs. administrator roles).
- **Standalone Deployment Only:** MediCore stores data in a local SQLite database file, making it unsuitable for multi-facility deployments where centralized data access is required. All data is stored on the local machine.
- **No Barcode/RFID Integration:** The system does not support barcode scanning or RFID-based stock tracking. All data entry is manual through the GUI.
- **No Automated Procurement:** The system generates alerts for low stock but does not automate purchase order creation or integrate with supplier systems.
- **No Backup and Recovery Module:** The application does not include built-in database backup or recovery functionality. Users must manually backup the SQLite database file.
- **Limited Report Customization:** Reports are generated with a fixed format. There is no option for users to customize report templates, select specific columns, or add custom filters beyond date range.
- **No Audit Log:** While stock movements are recorded, there is no dedicated audit log that captures all user actions including login/logout events, settings changes, and failed access attempts.
- **Internet Connectivity:** The application works entirely offline, which means there is no capability for cloud synchronization, remote monitoring, or web-based access to inventory data.

13. Future Enhancement

The following enhancements are planned for future versions of MediCore to expand its capabilities and address current limitations:

13.1 Cloud Database Integration

Replace or supplement the SQLite database with a cloud-based database (PostgreSQL or MySQL) to enable multi-facility access, real-time data synchronization, and remote monitoring of inventory across different locations.

13.2 Barcode Scanner Integration

Add support for USB barcode scanners to enable faster medicine lookup, stock entry, and dispensing operations. This would significantly reduce manual data entry errors and improve operational speed.

13.3 Automated Procurement and Supplier Integration

Implement an automated purchase order generation feature that creates procurement requests when stock falls below the threshold. Integration with supplier APIs or email-based order submission would streamline the restocking process.

13.4 Mobile Application

Develop a complementary Android/iOS mobile application for real-time stock monitoring and alert reception, allowing administrators to stay informed about inventory status while away from the desktop system.

13.5 Advanced Analytics and Forecasting

Enhance the dashboard with predictive analytics capabilities using machine learning algorithms to forecast medicine demand based on historical consumption patterns, helping administrators proactively manage stock levels.

14. Conclusion

MediCore Hospital Admin System was successfully developed as a standalone JavaFX and SQLite application for improving medicine inventory management in small and medium healthcare facilities. The system delivers secure administrator access, medicine CRUD operations, stock IN and stock OUT processing, low-stock and near-expiry alerts, dashboard analytics, and CSV/PDF report generation through a simple desktop workflow.

By replacing manual record keeping with a structured digital solution, MediCore improves accuracy, visibility, and reporting readiness while remaining lightweight and easy to deploy. The project also demonstrates the practical use of JavaFX, SQLite, repository-based data access, and report generation in a real healthcare scenario. Overall, the system meets its core objectives and provides a strong foundation for future enhancements such as multi-user access, barcode integration, automated backup, and cloud connectivity.

15. Bibliography

The following references were consulted during the design, development, and documentation of the MediCore Medical Inventory Management System:

Books

- Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley Professional.
- Deitel, P., & Deitel, H. (2020). Java: How to Program, Early Objects (11th ed.). Pearson Education.
- Connolly, T., & Begg, C. (2015). Database Systems: A Practical Approach to Design, Implementation, and Management (6th ed.). Pearson Education.

Online Documentation

- Oracle Corporation. (2024). Java SE 24 Documentation. Retrieved from <https://docs.oracle.com/en/java/javase/24/>
- OpenJFX. (2024). JavaFX 21 API Documentation. Retrieved from <https://openjfx.io/javadoc/21/>
- Xerial. (2024). SQLite JDBC Driver Documentation. Retrieved from <https://github.com/xerial/sqlite-jdbc>